File System (fs) module - reading and writing files

Absolutely! Here's a detailed explanation of the **File System (`fs`) module** in Node.js, focusing on **reading and writing files**.

📂 Node.js `fs` Module – Overview

The `fs` (File System) module allows you to **interact with the file system**, enabling operations like reading, writing, updating, deleting files, and more.

It comes in two main API styles:

- **Synchronous** (blocking)
- **Asynchronous** (non-blocking, callback-based or Promise-based)

🔷 1. Importing `fs`

```
// CommonJS
const fs = require('fs');

// ES Module
import fs from 'fs'; // Requires "type": "module" in package.json
```

📖 2. Reading Files

✅ Asynchronous (Preferred)

```
fs.readFile('example.txt', 'utf8', (err, data) ⇒ {
  if (err) {
    console.error('Error reading file:', err);
    return;
```

```
  }
  console.log('File content:', data);
});
```

✅ Synchronous (Blocking)

```
const data = fs.readFileSync('example.txt', 'utf8');
console.log('File content:', data);
```

✅ Promise-based (Modern)

```
const fsPromises = require('fs').promises;

async function readFile() {
  try {
    const data = await fsPromises.readFile('example.txt', 'utf8');
    console.log('File content:', data);
  } catch (err) {
    console.error('Error:', err);
  }
}

readFile();
```

✍️ 3. Writing Files

✅ Asynchronous

```
fs.writeFile('output.txt', 'Hello, World!', (err) ⇒ {
  if (err) {
    console.error('Error writing file:', err);
    return;
  }
```

```
  console.log('File written successfully.');
});
```

✅ Synchronous

```
fs.writeFileSync('output.txt', 'Hello, World!');
console.log('File written successfully.');
```

✅ Promise-based

```
async function writeFile() {
  try {
    await fsPromises.writeFile('output.txt', 'Hello, Promises!');
    console.log('File written successfully.');
  } catch (err) {
    console.error('Error:', err);
  }
}

writeFile();
```

📝 4. Appending to a File

Async

```
fs.appendFile('output.txt', '\nAppended text.', (err) => {
  if (err) throw err;
  console.log('Appended successfully.');
});
```

🚫 5. Deleting a File

```
fs.unlink('output.txt', (err) => {
  if (err) throw err;
  console.log('File deleted.');
});
```

🧠 Key Notes

- Always prefer **asynchronous methods** in production to avoid blocking the event loop.
- Use **Promise-based** `fs.promises` for cleaner async/await syntax.
- File paths can be **relative or absolute**.
- Combine with `path` module for OS-independent paths.

📁 Example Use Case

```
const fs = require('fs');

fs.writeFile('notes.txt', 'My first note', (err) => {
  if (err) return console.error(err);

  fs.readFile('notes.txt', 'utf8', (err, data) => {
    if (err) return console.error(err);
    console.log('Read from file:', data);
  });
});
```

Let me know if you'd like to learn about watching files, working with directories, or using streams with `fs`.